FINAL-YEAR PROJECT

Enterprise AI Knowledge Assistant

PDF Retrieval-Augmented Generation Capstone

Combined Project Report and Complete Source Listing

Submission detail	Value
Student name	_____
Institution / department	_____
Course / programme	_____
Supervisor	_____
Submission date	_____

Prepared from the project files currently present in the workspace. Complete the institutional details above and check departmental formatting requirements before submitting.

Abstract

This submission brings together six standalone Python assignment implementations and a PDF-based AI knowledge assistant capstone. The assignments demonstrate REST CRUD, database persistence, bearer-token authentication, Docker-based full-stack packaging, an AI-provider API integration, and a robots-aware web scraper. The capstone accepts PDF documents, extracts and chunks text, embeds chunks into a persistent Chroma vector store, retrieves relevant passages, and generates grounded answers with source/page references through a Streamlit interface.

The codebase includes local setup instructions, dependency manifests, Docker configuration for the capstone, upload validation, a final-submission checklist, and focused offline unit tests. The assignment brief supplied to the project contained titles rather than detailed rubrics; accordingly, the six assignments are reasonable educational implementations of those titles and should be compared with the actual course requirements before submission.

1. Project context and objectives

The project develops practical backend and AI engineering skills through small, independently runnable applications, culminating in a Retrieval-Augmented Generation (RAG) document question-answering system. RAG combines document retrieval with language generation: the system finds relevant passages from a user-provided collection and supplies them as context to a language model.

- Build and document six independent examples matching the supplied assignment topics.
- Provide a reproducible PDF ingestion, chunking, embedding, retrieval, and question-answering pipeline.
- Expose the capstone through a Streamlit interface with additive PDF upload and source citations.
- Validate input, keep secrets out of source and archives, and document limitations and test evidence.

2. Assignment portfolio

No.	Assignment / implementation	Main technologies and behavior
1	CRUD REST API	FastAPI; in-memory item create/read/update/delete; input validation; not persistent across restarts.
2	Backend connected to a database	FastAPI, SQLAlchemy 2, SQLite by default; persistent CRUD endpoint implementation.
3	Endpoint authentication	OAuth2 password form, environment-provided credentials, short-lived HS256 JWT bearer tokens, protected route.
4	Containerized full stack	FastAPI, server-rendered Jinja2 inventory UI, SQLite, Dockerfile, Compose named volume.
5	AI API integration	FastAPI endpoint calling OpenAI Chat Completions; environment-based key and explicit provider errors.
6	Respectful web scraper	Requests and BeautifulSoup; robots.txt checks, same-origin crawl, delay, page cap, timeout, 429 handling.
7	Enterprise AI knowledge assistant capstone	PDF parsing and chunking, OpenAI embeddings/chat, Chroma persistence, MMR retrieval, Streamlit upload and Q&A.;

3. Capstone architecture

The capstone is organized around a shared core module so both the command-line scripts and Streamlit interface use the same ingestion, validation, indexing, and QA logic.

Stage	Implementation
Document input	PDF files in docs/ or uploaded from the Streamlit sidebar; uploads are additive.
Validation and persistence	PDF extension/header/readability checks, 20 MB limit, safe filename, content hash, local upload storage.
Extraction and chunking	PyPDFLoader extracts pages; RecursiveCharacterTextSplitter uses 800-character chunks with 100-character overlap.
Embedding and storage	OpenAI text-embedding-3-small embeddings stored in persistent Chroma collection my_docs; stable IDs make repeat indexing idempotent.
Retrieval	Maximal Marginal Relevance, k=5 and fetch_k=20. Retrieved chunks keep filename and zero-based source page metadata.
Answer generation	gpt-4o-mini at temperature 0 is instructed to use only retrieved context and say it does not know when context is insufficient.
User interface	Streamlit chat renders answer and one-based source page references; cache is invalidated after successful uploads.

Data flow: PDF input -> validation -> page text extraction -> chunking -> embeddings -> Chroma index -> retrieval -> context-grounded answer -> source citations.

4. PDF upload workflow

- Users can select one or more PDF files in the Streamlit sidebar.
- The application rejects empty files, files over 20 MB, non-PDF extensions, invalid PDF headers, unreadable PDFs, and zero-page documents.
- Uploads are given a content-hash-prefixed safe filename and saved in docs/uploads/. An uploaded document is added to, rather than replacing, the knowledge base.
- Extractable PDF text is chunked and inserted into the current persistent Chroma collection. Scanned/image-only PDFs need OCR, which is not included.
- On successful indexing the cached QA chain is cleared; the app then exposes the updated collection to questions.

5. Running the capstone

From the P7_RAG_Document_QA directory, create a Python 3.10+ environment, install requirements.txt, and set OPENAI_API_KEY in the current shell. Launch with streamlit run streamlit_app.py. Upload PDFs through the sidebar. The optional command-line pipeline is: python 01_ingest_chunk.py; python 02_embed_store.py; python 03_qa_chain.py "What are the main topics?".

For Docker, set OPENAI_API_KEY and run docker compose up --build; open http://localhost:8501. Compose uses named volumes for documents and vectors so they persist through container restarts. The root project folders assignment_01_crud_api through assignment_06_respectful_scraper each contain an independent README, requirements.txt, and runnable implementation.

6. Verification evidence

Check	Observed result
Python source diagnostics	No Pylance errors reported on the capstone core, UI, tests, and pipeline scripts at the time checked.
Offline unit tests	7 tests passed: chunk metadata/chunk splitting, invalid chunk settings, upload validation/storage/indexing, upload size rejection, and missing API key.
Streamlit startup smoke test	Server started and returned the Streamlit page HTML over localhost.
Live provider-backed end-to-end Q&A;	Not verified: it requires a valid OpenAI key and a real PDF document.
Assignment-specific automated tests	No dedicated automated test suite was included for assignments 1?6.

The reported checks describe the workspace validation performed; they are not a substitute for running the application against the student?s own PDF, API key, Docker environment, or course rubric.

7. Security, privacy, and responsible use

• Never include a real API key, .env file, private PDF, local Chroma database, or personal upload in a submission archive.

• PDF text is sent to the configured OpenAI provider for embeddings and generation. Confirm organizational and data-owner approval before indexing sensitive material.

• The capstone has no login or authorization. Do not expose it to an untrusted network or use it as a production multi-user service.

• Use the scraper only on sites and pages you are authorized to access. Respect robots.txt, site terms, rate limits, and access restrictions; do not evade authentication or anti-bot controls.

• The JWT assignment demonstrates token validation but uses one environment-configured account; it is not a user management or production identity system.

8. Limitations and future work

• Add institutional user authentication, document-level access controls, tenant isolation, audit logging, and managed encryption before production use.

• Add OCR for scanned PDFs and background processing for large collections.

• Add a repeatable retrieval/answer evaluation dataset, precision and recall measurements, latency/cost measurements, and broader integration tests.

• Add deletion/re-index management for user documents and better reporting for corrupt or partially extractable PDFs.

• Expand the assignment examples against the actual course rubrics; only assignment titles were provided in this workspace.

9. Conclusion

The portfolio demonstrates a progression from API and database fundamentals to authentication, containerization, external AI integration, and respectful scraping. The capstone combines document processing, vector search, language-model generation, and an interactive upload-enabled UI. It is packaged with setup documentation, tests, Docker configuration, and a submission checklist while explicitly documenting remaining production and evaluation work.

Submission notes

Before final submission, fill in institution-specific details, run the tests and app on the submission machine, perform an end-to-end check with an authorized PDF and valid API key, verify citations, and follow the department?s rules for AI assistance and source attribution. Do not represent unmeasured performance as a result.

Appendix A ? Complete project files

The following appendix reproduces 39 current text/source/configuration files found in the six assignment directories, the capstone directory, and the top-level assignment brief. Local virtual environments, generated databases/vector indexes, user PDFs, compiled caches, secrets, existing ZIP archives, and this generated PDF are excluded.

assignment_01_crud_api/main.py

```
"""Small in-memory CRUD API for the assignment's Item resource."""

from itertools import count
from threading import Lock

from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, Field

app = FastAPI(title="Assignment 1 - CRUD API", version="1.0.0")


class ItemCreate(BaseModel):
    name: str = Field(min_length=1, max_length=120)
    description: str = Field(default="", max_length=1000)


class Item(ItemCreate):
    id: int


_items: dict[int, Item] = {}
_ids = count(1)
_lock = Lock()


@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}


@app.post("/items", response_model=Item, status_code=status.HTTP_201_CREATED)
def create_item(payload: ItemCreate) -> Item:
    with _lock:
        item = Item(id=next(_ids), **payload.model_dump())
        _items[item.id] = item
        return item


@app.get("/items", response_model=list[Item])
def list_items() -> list[Item]:
    with _lock:
        return list(_items.values())


@app.get("/items/{item_id}", response_model=Item)
def get_item(item_id: int) -> Item:
    with _lock:
        item = _items.get(item_id)
    if item is None:
        raise HTTPException(status_code=404, detail="Item not found")
    return item


@app.put("/items/{item_id}", response_model=Item)
def update_item(item_id: int, payload: ItemCreate) -> Item:
    with _lock:
        if item_id not in _items:
            raise HTTPException(status_code=404, detail="Item not found")
        item = Item(id=item_id, **payload.model_dump())
        _items[item_id] = item
        return item


@app.delete("/items/{item_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_item(item_id: int) -> None:
    with _lock:
        if _items.pop(item_id, None) is None:
            raise HTTPException(status_code=404, detail="Item not found")
```

assignment_01_crud_api/README.md

````
# Assignment 1 — CRUD API

A standalone FastAPI service with create, read, update, and delete operations
for items. Data is held in memory and resets when the process restarts.

## Run

```powershell
py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn main:app --reload
```

Open `http://127.0.0.1:8000/docs` for the interactive API.

| Method | Route | Result |
|---|---|---|
| `POST` | `/items` | Create an item |
| `GET` | `/items` | List all items |
| `GET` | `/items/{item_id}` | Read one item |
| `PUT` | `/items/{item_id}` | Replace an item |
| `DELETE` | `/items/{item_id}` | Delete an item |

The in-memory store is intentional here; database persistence is covered by
Assignment 2.
````

assignment_01_crud_api/requirements.txt

```
fastapi>=0.110,<1
uvicorn[standard]>=0.29,<1
```

assignment_02_database_api/main.py

```
"""CRUD API backed by a local SQLite database using SQLAlchemy 2."""

import os
from collections.abc import Generator

from fastapi import Depends, FastAPI, HTTPException, status
from pydantic import BaseModel, ConfigDict, Field
from sqlalchemy import String, create_engine, select
from sqlalchemy.orm import DeclarativeBase, Mapped, Session, mapped_column, sessionmaker

DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./items.db")
_connect_args = {"check_same_thread": False} if DATABASE_URL.startswith("sqlite") else {}
engine = create_engine(DATABASE_URL, connect_args=_connect_args)
SessionLocal = sessionmaker(bind=engine, expire_on_commit=False)
app = FastAPI(title="Assignment 2 - Database API", version="1.0.0")


class Base(DeclarativeBase):
    pass


class ItemRecord(Base):
    __tablename__ = "items"

    id: Mapped[int] = mapped_column(primary_key=True, autoincrement=True)
    name: Mapped[str] = mapped_column(String(120), nullable=False)
    description: Mapped[str] = mapped_column(String(1000), nullable=False, default="")


class ItemCreate(BaseModel):
    name: str = Field(min_length=1, max_length=120)
    description: str = Field(default="", max_length=1000)


class ItemRead(ItemCreate):
    model_config = ConfigDict(from_attributes=True)

    id: int


@app.on_event("startup")
def create_tables() -> None:
    Base.metadata.create_all(engine)


def get_session() -> Generator[Session, None, None]:
    with SessionLocal() as session:
        yield session


@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}


@app.post("/items", response_model=ItemRead, status_code=status.HTTP_201_CREATED)
def create_item(
    payload: ItemCreate, session: Session = Depends(get_session)
) -> ItemRecord:
    item = ItemRecord(**payload.model_dump())
    session.add(item)
    session.commit()
    session.refresh(item)
    return item


@app.get("/items", response_model=list[ItemRead])
def list_items(session: Session = Depends(get_session)) -> list[ItemRecord]:
    return list(session.scalars(select(ItemRecord).order_by(ItemRecord.id)))


@app.get("/items/{item_id}", response_model=ItemRead)
def get_item(item_id: int, session: Session = Depends(get_session)) -> ItemRecord:
    item = session.get(ItemRecord, item_id)
    if item is None:
        raise HTTPException(status_code=404, detail="Item not found")
    return item


@app.put("/items/{item_id}", response_model=ItemRead)
def update_item(
    item_id: int, payload: ItemCreate, session: Session = Depends(get_session)
) -> ItemRecord:
    item = session.get(ItemRecord, item_id)
    if item is None:
        raise HTTPException(status_code=404, detail="Item not found")
    for field, value in payload.model_dump().items():
```

```
        setattr(item, field, value)
    session.commit()
    session.refresh(item)
    return item


@app.delete("/items/{item_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_item(item_id: int, session: Session = Depends(get_session)) -> None:
    item = session.get(ItemRecord, item_id)
    if item is None:
        raise HTTPException(status_code=404, detail="Item not found")
    session.delete(item)
    session.commit()
```

assignment_02_database_api/README.md

```
# Assignment 2 — Connect a backend to a database

This independent FastAPI project stores items in SQLite through SQLAlchemy 2.
The database file is created on first startup.

## Run

```powershell
py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn main:app --reload
```

Browse to `http://127.0.0.1:8000/docs`. Override `DATABASE_URL` to use another
SQLAlchemy-supported database; install that database's driver separately.
SQLite is the default for a zero-service local demo.

The API exposes `POST`, `GET`, `PUT`, and `DELETE` at `/items`.
```

assignment_02_database_api/requirements.txt

```
fastapi>=0.110,<1
uvicorn[standard]>=0.29,<1
sqlalchemy>=2.0,<3
```

assignment_03_auth_api/main.py

```
"""JWT-protected API with credentials and signing key supplied by environment."""

import hashlib
import hmac
import os
from datetime import datetime, timedelta, timezone

import jwt
from fastapi import Depends, FastAPI, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from pydantic import BaseModel

app = FastAPI(title="Assignment 3 - Protected API", version="1.0.0")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/token")
ALGORITHM = "HS256"
TOKEN_LIFETIME_MINUTES = 30


class TokenResponse(BaseModel):
    access_token: str
    token_type: str = "bearer"


def _required_setting(name: str) -> str:
    value = os.getenv(name)
    if not value:
        raise HTTPException(
            status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
            detail=f"Server configuration is missing {name}",
        )
    return value


def _password_digest(password: str, salt: bytes) -> bytes:
    return hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, 310_000)


def authenticate(username: str, password: str) -> bool:
    configured_username = _required_setting("API_USERNAME")
    configured_password = _required_setting("API_PASSWORD")
    salt = _required_setting("PASSWORD_SALT").encode("utf-8")
    username_matches = hmac.compare_digest(username, configured_username)
    password_matches = hmac.compare_digest(
        _password_digest(password, salt),
        _password_digest(configured_password, salt),
    )
    return username_matches and password_matches


def issue_token(username: str) -> str:
    secret = _required_setting("JWT_SECRET")
    expires_at = datetime.now(timezone.utc) + timedelta(
        minutes=TOKEN_LIFETIME_MINUTES
    )
    return jwt.encode(
        {"sub": username, "exp": expires_at},
        secret,
        algorithm=ALGORITHM,
    )


@app.post("/token", response_model=TokenResponse)
def login(form: OAuth2PasswordRequestForm = Depends()) -> TokenResponse:
    if not authenticate(form.username, form.password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )
    return TokenResponse(access_token=issue_token(form.username))


def get_current_user(token: str = Depends(oauth2_scheme)) -> str:
    try:
        payload = jwt.decode(
            token,
            _required_setting("JWT_SECRET"),
            algorithms=[ALGORITHM],
            options={"require": ["exp", "sub"]},
        )
        subject = payload.get("sub")
        if not isinstance(subject, str) or not subject:
            raise jwt.InvalidTokenError("Missing subject")
        return subject
    except jwt.PyJWTError as exc:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
```

```
            detail="Invalid or expired bearer token",
            headers={"WWW-Authenticate": "Bearer"},
        ) from exc


@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}


@app.get("/protected")
def protected(current_user: str = Depends(get_current_user)) -> dict[str, str]:
    return {"message": f"Authenticated as {current_user}"}
```

assignment_03_auth_api/README.md

````
# Assignment 3 — Protect endpoints with authentication

This API issues short-lived HS256 bearer tokens from `/token` and protects
`/protected`. It uses a single environment-configured account for a compact
demonstration; it is not a multi-user identity system.

## Configure and run (PowerShell)

Use unique values; do not commit the secrets:

```powershell
$env:API_USERNAME = "student"
$env:API_PASSWORD = "replace-with-a-strong-password"
$env:PASSWORD_SALT = "replace-with-a-random-salt"
$env:JWT_SECRET = "replace-with-at-least-32-random-bytes"
py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn main:app --reload
```

Missing configuration produces an explicit server error instead of silently
using a known development secret. Obtain a token in `/docs` by calling
`POST /token` with form fields `username` and `password`, then authorize with
the returned bearer token.
````

assignment_03_auth_api/requirements.txt

```
fastapi>=0.110,<1
uvicorn[standard]>=0.29,<1
PyJWT>=2.8,<3
python-multipart>=0.0.9,<1
```

assignment_04_docker_fullstack/compose.yaml

```
services:
  web:
    build: .
    ports:
      - "8000:8000"
    volumes:
      - inventory_data:/app/data
    restart: unless-stopped

volumes:
  inventory_data:
```

assignment_04_docker_fullstack/Dockerfile

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    DATABASE_PATH=/app/data/items.db

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY main.py .
COPY templates ./templates

RUN useradd --create-home --uid 10001 appuser \
    && mkdir -p /app/data \
    && chown -R appuser:appuser /app
USER appuser

EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

assignment_04_docker_fullstack/main.py

```
"""A small server-rendered inventory app with a persistent SQLite database."""

import os
import sqlite3
from contextlib import asynccontextmanager
from pathlib import Path
from typing import Annotated

from fastapi import FastAPI, Form, Request
from fastapi.responses import HTMLResponse, RedirectResponse
from fastapi.templating import Jinja2Templates

BASE_DIR = Path(__file__).resolve().parent
DATABASE_PATH = Path(os.getenv("DATABASE_PATH", str(BASE_DIR / "data" / "items.db")))
templates = Jinja2Templates(directory=str(BASE_DIR / "templates"))


def connect() -> sqlite3.Connection:
    DATABASE_PATH.parent.mkdir(parents=True, exist_ok=True)
    connection = sqlite3.connect(DATABASE_PATH)
    connection.row_factory = sqlite3.Row
    return connection


@asynccontextmanager
async def lifespan(_: FastAPI):
    with connect() as connection:
        connection.execute(
            """
            CREATE TABLE IF NOT EXISTS items (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                name TEXT NOT NULL CHECK(length(name) BETWEEN 1 AND 120),
                description TEXT NOT NULL DEFAULT ''
                    CHECK(length(description) <= 1000)
            )
            """
        )
    yield


app = FastAPI(title="Assignment 4 - Docker Full Stack", lifespan=lifespan)


@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}


@app.get("/", response_class=HTMLResponse)
def index(request: Request):
    with connect() as connection:
        items = connection.execute("SELECT id, name, description FROM items ORDER BY id").fetchall()
    return templates.TemplateResponse(
        request=request,
        name="index.html",
        context={"items": items},
    )


@app.post("/items")
def create_item(
    name: Annotated[str, Form(min_length=1, max_length=120)],
    description: Annotated[str, Form(max_length=1000)] = "",
) -> RedirectResponse:
    with connect() as connection:
        connection.execute(
            "INSERT INTO items (name, description) VALUES (?, ?)",
            (name.strip(), description.strip()),
        )
    return RedirectResponse("/", status_code=303)


@app.post("/items/{item_id}/delete")
def delete_item(item_id: int) -> RedirectResponse:
    with connect() as connection:
        connection.execute("DELETE FROM items WHERE id = ?", (item_id,))
    return RedirectResponse("/", status_code=303)
```

assignment_04_docker_fullstack/README.md

````
# Assignment 4 — Containerize a full stack

This project includes a FastAPI backend, server-rendered Jinja2 frontend, and
SQLite persistence. Docker Compose builds and runs the app; a named volume keeps
the database when the container is replaced.

## Run with Docker Compose

From this directory:

```powershell
docker compose up --build
```

Visit `http://localhost:8000`; use `docker compose down` to stop. The named
volume is preserved by default. To delete it, explicitly run
`docker compose down --volumes`.

## Run without Docker

```powershell
py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn main:app --reload
```

The UI adds and deletes inventory items. `/health` is available for a basic
container health check.
````

assignment_04_docker_fullstack/requirements.txt

```
fastapi>=0.110,<1
uvicorn[standard]>=0.29,<1
jinja2>=3.1,<4
python-multipart>=0.0.9,<1
```

assignment_04_docker_fullstack/templates/index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Inventory</title>
    <style>
      body { font: 16px/1.5 system-ui, sans-serif; margin: 2rem auto; max-width: 48rem; padding: 0 1rem; }
      label, input, textarea { display: block; width: 100%; }
      input, textarea { box-sizing: border-box; margin: .3rem 0 1rem; padding: .6rem; }
      button { padding: .5rem .8rem; }
      li { border-top: 1px solid #ddd; padding: 1rem 0; }
      form.delete { display: inline; }
    </style>
  </head>
  <body>
    <h1>Inventory</h1>
    <form method="post" action="/items">
      <label for="name">Name</label>
      <input id="name" name="name" minlength="1" maxlength="120" required>
      <label for="description">Description</label>
      <textarea id="description" name="description" maxlength="1000"></textarea>
      <button type="submit">Add item</button>
    </form>
    <h2>Items</h2>
    {% if items %}
      <ul>
        {% for item in items %}
          <li>
            <strong>{{ item.name }}</strong>
            <p>{{ item.description }}</p>
            <form class="delete" method="post" action="/items/{{ item.id }}/delete">
              <button type="submit">Delete</button>
            </form>
          </li>
        {% endfor %}
      </ul>
    {% else %}
      <p>No items yet.</p>
    {% endif %}
  </body>
</html>
```

assignment_05_ai_api/main.py

```
"""A small HTTP service that delegates a single prompt to OpenAI."""

import os

from fastapi import FastAPI, HTTPException, status
from openai import APIError, OpenAI
from pydantic import BaseModel, Field

app = FastAPI(title="Assignment 5 - AI API", version="1.0.0")


class PromptRequest(BaseModel):
    prompt: str = Field(min_length=1, max_length=8000)


class PromptResponse(BaseModel):
    model: str
    response: str


@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}


@app.post("/generate", response_model=PromptResponse)
def generate(payload: PromptRequest) -> PromptResponse:
    api_key = os.getenv("OPENAI_API_KEY")
    if not api_key:
        raise HTTPException(
            status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
            detail="OPENAI_API_KEY is not configured",
        )

    model = os.getenv("OPENAI_MODEL", "gpt-4o-mini")
    try:
        result = OpenAI(api_key=api_key).chat.completions.create(
            model=model,
            messages=[{"role": "user", "content": payload.prompt}],
            max_tokens=1000,
            temperature=0.2,
        )
    except APIError as exc:
        raise HTTPException(
            status_code=status.HTTP_502_BAD_GATEWAY,
            detail=f"AI provider request failed: {exc.message}",
        ) from exc

    answer = result.choices[0].message.content
    if not answer:
        raise HTTPException(
            status_code=status.HTTP_502_BAD_GATEWAY,
            detail="AI provider returned an empty response",
        )
    return PromptResponse(model=result.model, response=answer)
```

assignment_05_ai_api/README.md

````
# Assignment 5 — Integrate an AI API into a working service

The FastAPI endpoint `/generate` validates a prompt and returns text from the
OpenAI Chat Completions API. The API key is read only from the environment.

## Run (PowerShell)

```powershell
$env:OPENAI_API_KEY = "your-key"
# Optional: $env:OPENAI_MODEL = "gpt-4o-mini"
py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn main:app --reload
```

Send `POST /generate` with JSON `{"prompt":"Explain a simple concept."}`.
Interactive docs are at `http://127.0.0.1:8000/docs`. Provider failures are
reported as HTTP 502; missing key configuration is HTTP 503.
````

assignment_05_ai_api/requirements.txt

```
fastapi>=0.110,<1
uvicorn[standard]>=0.29,<1
openai>=1.30,<3
```

assignment_06_respectful_scraper/README.md

````
# Assignment 6 — A scraper that respects the sites it collects from

The scraper is intentionally limited to one site origin, requires permission
from `robots.txt` when it is available, waits at least one second between page
requests (two seconds by default), caps pages, uses timeouts, skips non-HTML
content, and stops on HTTP 429. A failed robots.txt request fails closed.

Use it only on public pages you are authorized to access. Do not bypass access
controls, logins, CAPTCHAs, or site restrictions. Review the site's terms and
robots policy and identify the scraper with an honest contact before use.

## Run

```powershell
py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python scrape.py https://example.org --max-pages 5 --delay 2 --output results.json
```

Output is JSON containing the URL, page title, and extracted text. This is an
educational example, not a guarantee that a site's terms permit automated
collection.
````

assignment_06_respectful_scraper/requirements.txt

```
requests>=2.31,<3
beautifulsoup4>=4.12,<5
```

assignment_06_respectful_scraper/scrape.py

```
"""Small robots-aware scraper for explicitly configured public HTML pages."""

import argparse
import json
import logging
import time
from dataclasses import asdict, dataclass
from urllib.parse import urljoin, urlparse
from urllib.robotparser import RobotFileParser

import requests
from bs4 import BeautifulSoup
from requests import Response

USER_AGENT = "StudentRespectfulScraper/1.0 (+contact: student@example.invalid)"
REQUEST_TIMEOUT_SECONDS = 15
logger = logging.getLogger("respectful_scraper")


@dataclass
class Page:
    url: str
    title: str
    text: str


class RespectfulScraper:
    def __init__(self, delay: float = 2.0, max_pages: int = 10) -> None:
        if delay < 1.0:
            raise ValueError("delay must be at least one second")
        if max_pages < 1:
            raise ValueError("max_pages must be at least one")
        self.delay = delay
        self.max_pages = max_pages
        self.session = requests.Session()
        self.session.headers.update({"User-Agent": USER_AGENT})
        self._robots: dict[str, RobotFileParser] = {}
        self._last_request_at: float | None = None

    def _robots_for(self, url: str) -> RobotFileParser:
        parsed = urlparse(url)
        origin = f"{parsed.scheme}://{parsed.netloc}"
        if origin not in self._robots:
            robots_url = urljoin(origin, "/robots.txt")
            parser = RobotFileParser()
            parser.set_url(robots_url)
            try:
                response = self.session.get(
                    robots_url, timeout=REQUEST_TIMEOUT_SECONDS
                )
                if response.status_code == 404:
                    parser.parse([])
                else:
                    response.raise_for_status()
                    parser.parse(response.text.splitlines())
            except requests.RequestException as exc:
                logger.warning("Could not retrieve %s: %s", robots_url, exc)
                parser.disallow_all = True
            self._robots[origin] = parser
        return self._robots[origin]

    def _get(self, url: str) -> Response | None:
        if not self._robots_for(url).can_fetch(USER_AGENT, url):
            logger.warning("Blocked by robots.txt, skipping %s", url)
            return None
        if self._last_request_at is not None:
            wait = self.delay - (time.monotonic() - self._last_request_at)
            if wait > 0:
                time.sleep(wait)
        try:
            response = self.session.get(url, timeout=REQUEST_TIMEOUT_SECONDS)
            self._last_request_at = time.monotonic()
            if response.status_code == 429:
                logger.warning("Rate limited (HTTP 429), stopping at %s", url)
                return None
            response.raise_for_status()
            if "text/html" not in response.headers.get("Content-Type", "").lower():
                logger.warning("Non-HTML response, skipping %s", url)
                return None
            return response
        except requests.RequestException as exc:
            logger.warning("Request failed for %s: %s", url, exc)
            return None

    def scrape(self, start_url: str) -> list[Page]:
        if urlparse(start_url).scheme not in {"http", "https"}:
            raise ValueError("start_url must use HTTP or HTTPS")
```

```
        start = urlparse(start_url)
        seen: set[str] = set()
        pending = [start_url]
        pages: list[Page] = []
        while pending and len(pages) < self.max_pages:
            url = pending.pop(0)
            parsed = urlparse(url)
            if parsed.scheme not in {"http", "https"} or parsed.netloc != start.netloc:
                continue
            normalized = parsed._replace(fragment="").geturl()
            if normalized in seen:
                continue
            seen.add(normalized)
            response = self._get(normalized)
            if response is None:
                continue

            soup = BeautifulSoup(response.text, "html.parser")
            for unwanted in soup(["script", "style", "noscript", "svg"]):
                unwanted.decompose()
            title = soup.title.get_text(" ", strip=True) if soup.title else ""
            text = soup.get_text(" ", strip=True)
            pages.append(Page(url=normalized, title=title, text=text[:20_000]))

            for anchor in soup.find_all("a", href=True):
                candidate = urlparse(urljoin(normalized, anchor["href"]))
                if (
                    candidate.scheme in {"http", "https"}
                    and candidate.netloc == start.netloc
                ):
                    next_url = candidate._replace(fragment="").geturl()
                    if next_url not in seen and next_url not in pending:
                        pending.append(next_url)

        return pages


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("url", help="Start URL on a public site you are permitted to scrape")
    parser.add_argument("--delay", type=float, default=2.0, help="Minimum seconds between page requests")
    parser.add_argument("--max-pages", type=int, default=10)
    parser.add_argument("--output", default="scraped_pages.json")
    args = parser.parse_args()

    logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")
    pages = RespectfulScraper(delay=args.delay, max_pages=args.max_pages).scrape(args.url)
    with open(args.output, "w", encoding="utf-8") as output:
        json.dump([asdict(page) for page in pages], output, ensure_ascii=False, indent=2)
    print(f"Saved {len(pages)} page(s) to {args.output}")


if __name__ == "__main__":
    main()
```

assignments

```
1.building crud api from scratch
2.connect a backend to a database
3.protecting endpoints with authentication
4.containerize a full stack with docker
5.integrate an ai api into a working service
6.writing a web scraper that respects the sites it collects from
7.capstone" enterprise ai knowledge assistant(p7-rag-document-qa)"
```

ASSIGNMENTS_README.md

```
# Assignments and capstone

The assignment brief supplied for this workspace contains titles only. These
standalone examples turn those titles into small, runnable Python projects:

1. `assignment_01_crud_api` — in-memory CRUD REST API
2. `assignment_02_database_api` — CRUD API backed by SQLite and SQLAlchemy
3. `assignment_03_auth_api` — JWT-protected API
4. `assignment_04_docker_fullstack` — server-rendered web app and Docker Compose
5. `assignment_05_ai_api` — API service using the OpenAI chat API
6. `assignment_06_respectful_scraper` — robots-aware, rate-limited scraper
7. `P7_RAG_Document_QA` — PDF knowledge assistant capstone

Each assignment has its own README and dependency list. Run commands from that
assignment's directory. Add assignment-specific rubric requirements, if any,
before submitting; the implementations cannot infer requirements absent from
the supplied brief.
```

P7_RAG_Document_QA/.dockerignore

```
.venv
venv
__pycache__
.git
.env
.pytest_cache
tests
docs
chroma_db
*.db
```

P7_RAG_Document_QA/.env.example

```
# Copy to .env for local configuration. Never commit a real API key.
OPENAI_API_KEY=
```

P7_RAG_Document_QA/.gitignore

```
.venv/
venv/
__pycache__/
*.py[cod]
.pytest_cache/
.coverage
htmlcov/
.env
docs/
chroma_db/
*.db
.streamlit/secrets.toml
```

P7_RAG_Document_QA/.streamlit/config.toml

```
[server]
maxUploadSize = 20
headless = true
```

P7_RAG_Document_QA/01_ingest_chunk.py

```
"""Step 1: load PDFs from docs/ and split them into overlapping chunks."""

import argparse
from pathlib import Path

from rag_core import DEFAULT_DOCS_DIR, load_and_split_documents


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--docs", type=Path, default=DEFAULT_DOCS_DIR)
    args = parser.parse_args()

    chunks = load_and_split_documents(args.docs)
    print(f"Loaded and split documents into {len(chunks)} chunks")
    print(f"Sample: {chunks[0].page_content[:200]}")


if __name__ == "__main__":
    main()
```

P7_RAG_Document_QA/02_embed_store.py

```
"""Step 2: ingest PDFs, embed chunks, and upsert them into persistent Chroma."""

import argparse
from pathlib import Path

from rag_core import DEFAULT_DB_DIR, DEFAULT_DOCS_DIR, load_and_split_documents, store_chunks


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--docs", type=Path, default=DEFAULT_DOCS_DIR)
    parser.add_argument("--db", type=Path, default=DEFAULT_DB_DIR)
    args = parser.parse_args()

    chunks = load_and_split_documents(args.docs)
    vectorstore = store_chunks(chunks, args.db)
    print(f"Upserted {len(chunks)} chunks; collection now has {vectorstore._collection.count()}")


if __name__ == "__main__":
    main()
```

P7_RAG_Document_QA/03_qa_chain.py

```
"""Step 3: query the persistent RAG chain from the terminal."""

import argparse
from pathlib import Path

from rag_core import DEFAULT_DB_DIR, build_qa_chain


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument(
        "question",
        nargs="?",
        default="What are the main topics covered?",
    )
    parser.add_argument("--db", type=Path, default=DEFAULT_DB_DIR)
    args = parser.parse_args()

    result = build_qa_chain(args.db).invoke(args.question)
    print("Answer:", result["result"])
    print("Sources:")
    for document in result["source_documents"]:
        source = document.metadata.get("source", "unknown")
        page = int(document.metadata.get("page", 0)) + 1
        print(f"- {source}, page {page}")


if __name__ == "__main__":
    main()
```

P7_RAG_Document_QA/compose.yaml

```
services:
  rag-assistant:
    build: .
    ports:
      - "8501:8501"
    environment:
      OPENAI_API_KEY: ${OPENAI_API_KEY:-}
    volumes:
      - rag_documents:/app/docs
      - rag_vectors:/app/chroma_db
    restart: unless-stopped

volumes:
  rag_documents:
  rag_vectors:
```

P7_RAG_Document_QA/Dockerfile

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY rag_core.py streamlit_app.py ./
COPY 01_ingest_chunk.py 02_embed_store.py 03_qa_chain.py ./
COPY .streamlit ./.streamlit

RUN useradd --create-home --uid 10001 appuser \
    && mkdir -p /app/docs /app/chroma_db \
    && chown -R appuser:appuser /app
USER appuser

EXPOSE 8501
CMD ["streamlit", "run", "streamlit_app.py", "--server.address=0.0.0.0", "--server.port=8501"]
```

P7_RAG_Document_QA/package_submission.ps1

```
$ErrorActionPreference = "Stop"

$projectRoot = $PSScriptRoot
$outputPath = Join-Path (Split-Path -Parent $projectRoot) "P7_RAG_Document_QA_submission.zip"
$excludedDirectories = @(
    ".git",
    ".venv",
    "venv",
    "__pycache__",
    ".pytest_cache",
    "docs",
    "chroma_db",
    "htmlcov"
)
$excludedFiles = @(".env", "secrets.toml")

$packageFiles = foreach ($file in Get-ChildItem -LiteralPath $projectRoot -Recurse -File -Force) {
    $relativePath = $file.FullName.Substring($projectRoot.Length).TrimStart("\", "/")
    $pathParts = $relativePath -split "[\\/]"
    if ($pathParts | Where-Object { $excludedDirectories -contains $_ }) {
        continue
    }
    if ($excludedFiles -contains $file.Name -or $file.Extension -in @(".pyc", ".db")) {
        continue
    }
    $file
}

Add-Type -AssemblyName System.IO.Compression
Add-Type -AssemblyName System.IO.Compression.FileSystem
if (Test-Path -LiteralPath $outputPath) {
    Remove-Item -LiteralPath $outputPath -Force
}

$archive = [System.IO.Compression.ZipFile]::Open(
    $outputPath,
    [System.IO.Compression.ZipArchiveMode]::Create
)
try {
    foreach ($file in $packageFiles) {
        $relativePath = $file.FullName.Substring($projectRoot.Length).TrimStart("\", "/")
        $entryName = "P7_RAG_Document_QA/" + $relativePath.Replace("\", "/")
        [System.IO.Compression.ZipFileExtensions]::CreateEntryFromFile(
            $archive,
            $file.FullName,
            $entryName,
            [System.IO.Compression.CompressionLevel]::Optimal
        ) | Out-Null
    }
}
finally {
    $archive.Dispose()
}

Write-Output "Created submission archive: $outputPath"
```

P7_RAG_Document_QA/rag_core.py

```
"""Shared ingestion, vector-store, and retrieval-chain setup for the capstone."""

import hashlib
from io import BytesIO
import os
import re
from pathlib import Path
from typing import Any

from langchain_community.document_loaders import PyPDFLoader
from langchain_community.vectorstores import Chroma
from langchain_core.documents import Document
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableLambda
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter
from pypdf import PdfReader
from pypdf.errors import PdfReadError

DEFAULT_DOCS_DIR = Path("docs")
DEFAULT_DB_DIR = Path("chroma_db")
COLLECTION_NAME = "my_docs"
EMBEDDING_MODEL = "text-embedding-3-small"
CHAT_MODEL = "gpt-4o-mini"
CHUNK_SIZE = 800
CHUNK_OVERLAP = 100
MAX_PDF_UPLOAD_BYTES = 20 * 1024 * 1024


def split_documents(
    documents: list[Document],
    chunk_size: int = CHUNK_SIZE,
    chunk_overlap: int = CHUNK_OVERLAP,
) -> list[Document]:
    """Split loaded pages into chunks and normalize source metadata."""
    if chunk_size <= 0 or chunk_overlap < 0 or chunk_overlap >= chunk_size:
        raise ValueError("Require chunk_size > chunk_overlap >= 0")
    if not documents:
        raise ValueError("No pages could be loaded from the supplied PDFs.")
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        separators=["\n\n", "\n", ". ", " ", ""],
    )
    chunks = splitter.split_documents(documents)
    for chunk in chunks:
        source = Path(str(chunk.metadata.get("source", "")))
        try:
            chunk.metadata["source"] = str(source.resolve().relative_to(Path.cwd()))
        except ValueError:
            chunk.metadata["source"] = str(source.resolve())
    return chunks


def load_and_split_documents(
    docs_dir: Path = DEFAULT_DOCS_DIR,
    chunk_size: int = CHUNK_SIZE,
    chunk_overlap: int = CHUNK_OVERLAP,
) -> list[Document]:
    """Load PDFs recursively and split pages into overlapping chunks."""
    if chunk_size <= 0 or chunk_overlap < 0 or chunk_overlap >= chunk_size:
        raise ValueError("Require chunk_size > chunk_overlap >= 0")
    if not docs_dir.is_dir():
        raise FileNotFoundError(f"PDF directory does not exist: {docs_dir}")
    pdf_paths = sorted(docs_dir.rglob("*.pdf"))
    if not pdf_paths:
        raise FileNotFoundError(f"No PDF files found under: {docs_dir}")

    documents: list[Document] = []
    for pdf_path in pdf_paths:
        documents.extend(PyPDFLoader(str(pdf_path)).load())
    return split_documents(documents, chunk_size, chunk_overlap)


def index_uploaded_pdf(
    filename: str,
    content: bytes,
    docs_dir: Path = DEFAULT_DOCS_DIR,
    persist_directory: Path = DEFAULT_DB_DIR,
) -> tuple[str, int]:
    """Validate, persist, and index one uploaded PDF without replacing others."""
    if not content:
        raise ValueError("The uploaded PDF is empty.")
    if len(content) > MAX_PDF_UPLOAD_BYTES:
        raise ValueError("PDF exceeds the 20 MB upload limit.")
    if b"%PDF-" not in content[:1024]:
        raise ValueError("The uploaded file does not have a valid PDF header.")
```

```
    original_name = Path(filename.replace("\\", "/")).name
    if not original_name.lower().endswith(".pdf"):
        raise ValueError("Only files with a .pdf extension are accepted.")
    safe_name = re.sub(r"[^A-Za-z0-9._-]", "_", original_name)
    if not safe_name or safe_name.lower() == ".pdf":
        raise ValueError("The uploaded PDF must have a valid filename.")

    try:
        reader = PdfReader(BytesIO(content), strict=False)
    except PdfReadError as exc:
        raise ValueError("The uploaded file is not a readable PDF.") from exc
    if not reader.pages:
        raise ValueError("The uploaded PDF contains no pages.")

    digest = hashlib.sha256(content).hexdigest()
    upload_dir = docs_dir / "uploads"
    upload_dir.mkdir(parents=True, exist_ok=True)
    destination = upload_dir / f"{digest}-{safe_name}"
    created_file = False
    if destination.exists():
        if destination.read_bytes() != content:
            raise FileExistsError(f"An upload already exists at {destination}.")
    else:
        destination.write_bytes(content)
        created_file = True

    indexed = False
    try:
        pages = PyPDFLoader(str(destination)).load()
        chunks = split_documents(pages)
        if not chunks:
            raise ValueError(
                "No extractable text was found in the PDF. "
                "Scanned PDFs need OCR before they can be indexed."
            )
        store_chunks(chunks, persist_directory)
        indexed = True
    finally:
        if created_file and not indexed:
            destination.unlink(missing_ok=True)

    try:
        source = destination.resolve().relative_to(Path.cwd())
    except ValueError:
        source = destination.resolve()
    return source.as_posix(), len(chunks)


def get_vectorstore(
    persist_directory: Path = DEFAULT_DB_DIR,
) -> Chroma:
    """Open the persistent Chroma collection with the configured embeddings."""
    if not os.getenv("OPENAI_API_KEY"):
        raise RuntimeError("OPENAI_API_KEY is not set. Export it before indexing or querying PDFs.")
    return Chroma(
        persist_directory=str(persist_directory),
        embedding_function=OpenAIEmbeddings(model=EMBEDDING_MODEL),
        collection_name=COLLECTION_NAME,
    )


def store_chunks(
    chunks: list[Document],
    persist_directory: Path = DEFAULT_DB_DIR,
) -> Chroma:
    """Upsert chunks by a stable content-derived id to make reruns idempotent."""
    if not chunks:
        raise ValueError("No document chunks to store")
    vectorstore = get_vectorstore(persist_directory)
    ids = []
    for chunk in chunks:
        identity = "\0".join(
            (
                str(chunk.metadata.get("source", "")),
                str(chunk.metadata.get("page", "")),
                chunk.page_content,
            )
        )
        ids.append(hashlib.sha256(identity.encode("utf-8")).hexdigest())
    vectorstore.add_documents(chunks, ids=ids)
    return vectorstore


def build_qa_chain(
    persist_directory: Path = DEFAULT_DB_DIR,
) -> Any:
    """Build a question-answer chain that returns source documents with answers."""
    vectorstore = get_vectorstore(persist_directory)
    collection = vectorstore.get(limit=1)
```

```
if not collection.get("ids"):
    raise ValueError(
        f"No indexed documents found in {persist_directory}. "
        "Add PDFs to docs and run 02_embed_store.py first."
    )

retriever = vectorstore.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 5, "fetch_k": 20},
)
prompt = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            "Answer the question using only the supplied context. "
            'If the answer is not in the context, say "I don\'t know." '
            "Do not invent citations.",
        ),
        ("human", "Context:\n{context}\n\nQuestion: {question}"),
    ]
)
llm = ChatOpenAI(model=CHAT_MODEL, temperature=0)

def format_documents(documents: list[Document]) -> str:
    return "\n\n".join(
        f"[Source: {document.metadata.get('source', 'unknown')}, "
        f"page: {int(document.metadata.get('page', 0)) + 1}]\n"
        f"{document.page_content}"
        for document in documents
    )

def qa_step(question: str) -> dict[str, Any]:
    docs = retriever.invoke(question)
    context = format_documents(docs)
    answer = llm.invoke(prompt.format(context=context, question=question)).content
    return {"result": answer, "source_documents": docs}

return RunnableLambda(qa_step)
```

P7_RAG_Document_QA/README.md

````
# Enterprise AI Knowledge Assistant — PDF RAG capstone

A local Retrieval-Augmented Generation (RAG) application that indexes PDF
documents in a persistent Chroma vector store, retrieves relevant passages,
and answers questions with source-file and page citations. PDFs can be added
from the Streamlit sidebar; each upload is added to the existing collection.

## Features

- Recursive PDF ingestion and configurable-size text chunking
- Persistent local Chroma storage with stable chunk IDs
- OpenAI embeddings and chat completion, configured through environment
- MMR retrieval and answers grounded in retrieved context
- Source filename and page citations shown with each answer
- Additive PDF uploads with filename sanitization, PDF validation, and a 20 MB
  per-file limit
- Local Python and Docker Compose run options
- Unit tests for chunking, upload validation, storage paths, and configuration

## Requirements

- Python 3.10 or later (the Docker image uses Python 3.12)
- An OpenAI API key with access to the configured embedding and chat models
- PDFs that you are authorized to process

API calls may incur charges. Document text is sent to the configured model
provider for embeddings and answers. Do not upload confidential or regulated
documents unless your organization's policy permits this.

## Local setup (PowerShell)

Run from this project directory:

```powershell
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
$env:OPENAI_API_KEY = "your-key"
```

Do not commit API keys. `.env.example` is a placeholder template, not an
automatically loaded environment file. For PowerShell, set the key in the
current shell as shown above.

## Run

### Web application

```powershell
streamlit run streamlit_app.py
```

Open the displayed local URL, use the sidebar to select one or more PDFs, and
click **Add PDFs to knowledge base**. Uploaded files are checked, saved under
`docs/uploads/`, split into chunks, and indexed into the current Chroma
collection. Uploading an already-indexed file does not create duplicate chunks.
Ask a question in the chat box; each assistant response lists the retrieved
source files and one-based page numbers.

The Streamlit server enforces the configured 20 MB per-file upload size. The
same limit is checked in application code. The uploader accepts PDF files;
image-only/scanned PDFs require OCR and are not indexed as text.

### Command-line pipeline

The first three scripts are optional inspection and terminal workflows; they
are not required before using the upload-enabled app.

```powershell
python 01_ingest_chunk.py
python 02_embed_store.py
python 03_qa_chain.py "What are the main topics?"
```

`01_ingest_chunk.py` inspects the PDF chunks. `02_embed_store.py` embeds and
upserts all PDFs under `docs/`. `03_qa_chain.py` asks a single terminal
question. Run these commands from this directory so the default `docs/` and
`chroma_db/` paths resolve consistently.

## Docker Compose

Set the key in the current PowerShell session, then build and run:

```powershell
$env:OPENAI_API_KEY = "your-key"
docker compose up --build
```
````

````
Open `http://localhost:8501`. Compose mounts named volumes for both uploaded
PDFs and the vector database, so they persist across container restarts. Stop
with `docker compose down`; to explicitly remove persisted data too, use
`docker compose down --volumes`.

## Tests

Run the offline unit tests from the project directory:

```powershell
python -m unittest discover -s tests -v
```

The tests do not call OpenAI. To verify the full application, set a valid API
key, upload a PDF containing text, and compare returned answers and citations
with the document itself.

## Pipeline settings

- Chunk size: 800 characters
- Chunk overlap: 100 characters
- Chroma collection: `my_docs`
- Embedding model: `text-embedding-3-small`
- Chat model: `gpt-4o-mini`, temperature 0
- Retrieval: Maximal Marginal Relevance, `k=5`, `fetch_k=20`

The model is instructed to answer only from retrieved context and say "I don't
know" if the context does not provide an answer. Retrieval can still miss
relevant passages; validate answers and citations against the source PDF.

## Project files

- `rag_core.py` — loading, chunking, validation, vector-store, and QA logic
- `01_ingest_chunk.py`, `02_embed_store.py`, `03_qa_chain.py` — command-line
  pipeline steps
- `streamlit_app.py` — chat UI and PDF upload interface
- `tests/` — offline unit tests
- `Dockerfile`, `compose.yaml` — container deployment
- `SUBMISSION_CHECKLIST.md` — packaging and final verification checklist
- `package_submission.ps1` — creates a ZIP in the parent folder while excluding
  local PDFs, vector data, environments, databases, and secrets

To create the archive from PowerShell:

```powershell
.\package_submission.ps1
```

## Limitations and responsible use

This is a local academic proof of concept, not a production enterprise service.
It has no user login, tenant isolation, document-level authorization,
encryption-at-rest configuration, OCR, automated evaluation suite, or
background job queue. Anyone who can access the local app can use its
knowledge base. Do not expose it to an untrusted network or use it for private
content without adding appropriate controls.
````

P7_RAG_Document_QA/requirements.txt

```
langchain>=0.3,<1
langchain-community>=0.3,<1
langchain-openai>=0.2,<1
langchain-text-splitters>=0.3,<1
chromadb>=0.5,<2
pypdf>=5,<7
streamlit>=1.40,<2
```

P7_RAG_Document_QA/streamlit_app.py

```
"""Streamlit chat interface for the persistent PDF RAG collection."""

from pathlib import Path

import streamlit as st
from openai import OpenAIError

from rag_core import DEFAULT_DB_DIR, DEFAULT_DOCS_DIR, build_qa_chain, index_uploaded_pdf

st.set_page_config(page_title="Document Q&A", page_icon=" ")
st.title("Document Q&A with RAG")
st.caption("Answers are grounded in the indexed PDFs and include source pages.")


@st.cache_resource
def get_qa_chain(db_directory: str):
    return build_qa_chain(Path(db_directory))


with st.sidebar:
    st.header("Knowledge base")
    st.write("Upload PDFs to add them to the existing index.")
    with st.form("pdf_upload_form", clear_on_submit=True):
        uploaded_files = st.file_uploader(
            "Choose PDF files",
            type=["pdf"],
            accept_multiple_files=True,
            help="Maximum 20 MB per file. Uploaded files are stored locally.",
        )
        submit_uploads = st.form_submit_button("Add PDFs to knowledge base")

    if submit_uploads:
        if not uploaded_files:
            st.warning("Choose at least one PDF to upload.")
        else:
            notices: list[tuple[str, str]] = []
            for uploaded_file in uploaded_files:
                try:
                    source, chunk_count = index_uploaded_pdf(
                        uploaded_file.name,
                        uploaded_file.getvalue(),
                        DEFAULT_DOCS_DIR,
                        DEFAULT_DB_DIR,
                    )
                    notices.append(
                        ("success", f"Indexed {source} ({chunk_count} chunks).")
                    )
                except (ValueError, RuntimeError, OSError, OpenAIError) as exc:
                    notices.append(
                        ("error", f"Could not index {uploaded_file.name}: {exc}")
                    )

            if any(level == "success" for level, _ in notices):
                st.session_state.upload_notices = notices
                get_qa_chain.clear()
                st.rerun()
            for level, message in notices:
                if level == "error":
                    st.error(message)

for level, message in st.session_state.pop("upload_notices", []):
    if level == "success":
        st.success(message)
    else:
        st.error(message)

try:
    qa_chain = get_qa_chain(str(DEFAULT_DB_DIR))
except (FileNotFoundError, ValueError, RuntimeError) as exc:
    st.error(str(exc))
    st.info(
        "Set OPENAI_API_KEY, upload a PDF from the sidebar, or add PDFs under "
        "docs/ and run 02_embed_store.py."
    )
    st.stop()

if "history" not in st.session_state:
    st.session_state.history = []

for message in st.session_state.history:
    with st.chat_message(message["role"]):
        st.write(message["content"])
        if message.get("sources"):
            with st.expander("Sources"):
                for source in message["sources"]:
                    st.write(f"- {source}")

query = st.chat_input("Ask a question about your documents...")
```

```
if query:
    st.session_state.history.append({"role": "user", "content": query})
    with st.chat_message("user"):
        st.write(query)

    with st.chat_message("assistant"):
        with st.spinner("Searching your documents..."):
            result = qa_chain.invoke(query)
        answer = result["result"]
        sources = list(
            dict.fromkeys(
                f"{document.metadata.get('source', 'unknown')} "
                f"(page {int(document.metadata.get('page', 0)) + 1})"
                for document in result["source_documents"]
            )
        )
        st.write(answer)
        with st.expander("Sources"):
            for source in sources:
                st.write(f"- {source}")
    st.session_state.history.append(
        {"role": "assistant", "content": answer, "sources": sources}
    )
```

P7_RAG_Document_QA/SUBMISSION_CHECKLIST.md

```
# Final-year project submission checklist

## Before packaging

- [ ] Replace the author, institution, supervisor, course, and submission-date
  placeholders in any cover page or report required by your department.
- [ ] Keep only documents and API keys you are authorized to use. Do not include
  a real `.env`, `.streamlit/secrets.toml`, private PDF, or local vector database.
- [ ] Create a fresh Python environment and install `requirements.txt`.
- [ ] Run the automated tests with
  `python -m unittest discover -s tests -v`.
- [ ] Run the app, upload a permitted PDF, ask questions with a known answer, and
  verify each answer's source file and page against that PDF.
- [ ] Test an unsupported/non-PDF upload and a question not answered in the
  documents; record the observed behavior honestly in your report.
- [ ] If Docker is part of the submission, build and run with Compose and verify
  uploaded documents remain available after restarting the container.
- [ ] Include screenshots, measured results, and evaluation data only when you
  have actually collected them. Do not claim results this project has not
  measured.
- [ ] Follow your department's rules for AI assistance, citations, code
  attribution, report format, and archive naming.

## Recommended archive contents

Include source files, `requirements.txt`, `README.md`, this checklist, tests,
Docker files, and an `.env.example` with no secrets. Exclude `.venv/`, generated
`__pycache__/`, `docs/` containing user PDFs, `.env`, and `chroma_db/`.
```

P7_RAG_Document_QA/tests/test_rag_core.py

```
"""Focused unit tests for the capstone's document handling."""

import hashlib
import os
import unittest
from pathlib import Path
from tempfile import TemporaryDirectory
from unittest.mock import Mock, patch

from langchain_core.documents import Document

import rag_core


class DocumentChunkingTests(unittest.TestCase):
    def test_split_documents_keeps_page_and_source_metadata(self) -> None:
        document = Document(
            page_content="This sentence is long enough to make several chunks.",
            metadata={"source": "docs/example.pdf", "page": 2},
        )

        chunks = rag_core.split_documents([document], chunk_size=18, chunk_overlap=3)

        self.assertGreater(len(chunks), 1)
        self.assertTrue(all(chunk.metadata["page"] == 2 for chunk in chunks))
        self.assertTrue(all("example.pdf" in chunk.metadata["source"] for chunk in chunks))

    def test_split_documents_rejects_invalid_chunk_settings(self) -> None:
        with self.assertRaises(ValueError):
            rag_core.split_documents([], chunk_size=10, chunk_overlap=10)


class UploadTests(unittest.TestCase):
    @patch("rag_core.store_chunks")
    @patch("rag_core.PyPDFLoader")
    @patch("rag_core.PdfReader")
    def test_upload_safely_stores_and_indexes_pdf(
        self, reader: Mock, loader: Mock, store_chunks: Mock
    ) -> None:
        reader.return_value.pages = [object()]
        loader.return_value.load.return_value = [
            Document(
                page_content="Uploaded PDF contains useful project information.",
                metadata={"source": "uploaded.pdf", "page": 0},
            )
        ]
        content = b"%PDF-1.4 test bytes"

        with TemporaryDirectory() as temporary_directory:
            root = Path(temporary_directory)
            source, chunk_count = rag_core.index_uploaded_pdf(
                r"..\final report.pdf",
                content,
                docs_dir=root / "docs",
                persist_directory=root / "chroma_db",
            )

            stored_file = root / "docs" / "uploads" / (
                f"{hashlib.sha256(content).hexdigest()}-final_report.pdf"
            )
            self.assertTrue(stored_file.is_file())
            self.assertEqual(chunk_count, 1)
            self.assertEqual(Path(source), stored_file)
            store_chunks.assert_called_once()

    def test_upload_rejects_non_pdf_header(self) -> None:
        with TemporaryDirectory() as temporary_directory:
            with self.assertRaisesRegex(ValueError, "valid PDF header"):
                rag_core.index_uploaded_pdf(
                    "notes.pdf", b"not a PDF", docs_dir=Path(temporary_directory)
                )

    def test_upload_rejects_wrong_extension(self) -> None:
        with TemporaryDirectory() as temporary_directory:
            with self.assertRaisesRegex(ValueError, r"\.pdf extension"):
                rag_core.index_uploaded_pdf(
                    "notes.txt",
                    b"%PDF-1.4 test",
                    docs_dir=Path(temporary_directory),
                )

    def test_upload_rejects_oversized_file_before_writing(self) -> None:
        with TemporaryDirectory() as temporary_directory:
            docs_dir = Path(temporary_directory) / "docs"
            content = b"%PDF-" + b"x" * rag_core.MAX_PDF_UPLOAD_BYTES
            with self.assertRaisesRegex(ValueError, "20 MB"):
                rag_core.index_uploaded_pdf("large.pdf", content, docs_dir=docs_dir)
```

```
            self.assertFalse(docs_dir.exists())


class ConfigurationTests(unittest.TestCase):
    def test_vectorstore_requires_api_key(self) -> None:
        with patch.dict(os.environ, {}, clear=True):
            with self.assertRaisesRegex(RuntimeError, "OPENAI_API_KEY"):
                rag_core.get_vectorstore()


if __name__ == "__main__":
    unittest.main()
```